Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

По дисциплине: Контейнеризация и оркестрация приложений

Тема работы: Docker Compose: оркестрация сервисов с Redis-кэшированием

Обучающийся: Лясин Илья Игоревич

Номер ИСУ: 466570

Факультет: Факультет прикладной информатики

Группа: K3321

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах

Обучающийся _____ Лясин И. И.

дата, подпись

Руководитель _____ Аминов Н. С.

дата, подпись

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Ход выполнения **4**

1.1 Этап 1. Подготовка проекта для Docker Compose 4

1.2 Этап 2. Добавление Redis для кэширования погоды 6

1.3 Этап 3. Масштабирование сервиса 10

1.4 Этап 4. Финальный артефакт 12

ЗАКЛЮЧЕНИЕ **13**

ВВЕДЕНИЕ

Цель работы: перенести запуск приложения, PostgreSQL и Redis в Docker Compose и освоить масштабирование сервисов.

В работе используются следующие инструменты: Docker, Docker Compose, PostgreSQL, Redis, а также приложение-ассистент, обращающееся к внешнему API погоды. Персональные параметры приведены в таблице 1.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	466570
Персональный порт (PostgreSQL, `PG_PORT`)	5500
Персональный порт (Redis, `REDIS_PORT`)	6440
Персональный порт (приложение, `APP_PORT`)	8070
Персональный префикс	`lyasin`
Секретный ключ	`secret70lyasin`
Имя базы данных	`lyasin70`

Результат хранится в ветке `lab-4` форка репозитория: https://github.com/7eliassen/course-containerization-orchestration-application/tree/lab-4.

1 Ход выполнения

1.1 Этап 1. Подготовка проекта для Docker Compose

Создадим файл `.env` с переменными окружения проекта (рисунок 1). Создадим файл `compose.yml` в корне проекта: в нём описаны два сервиса — база данных и приложение; переменные окружения подтягиваются из `.env` (рисунок 2). Далее поднимем его командой `docker compose up -d`

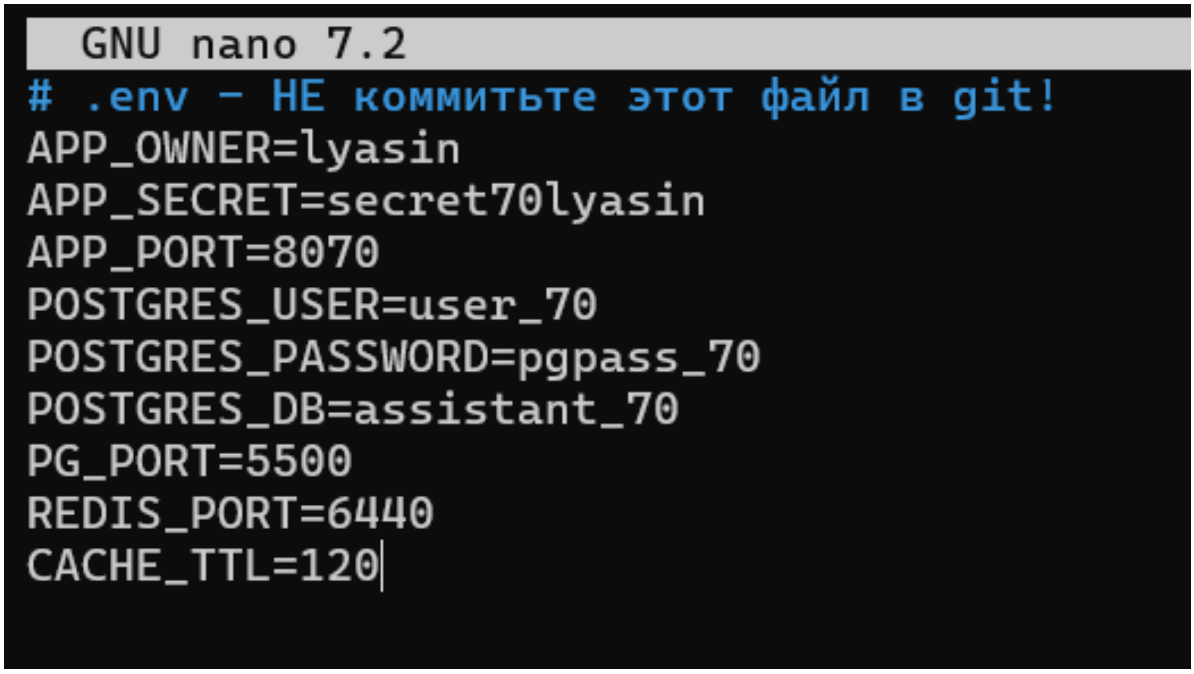


Рисунок 1 — Файл `.env` с переменными окружения

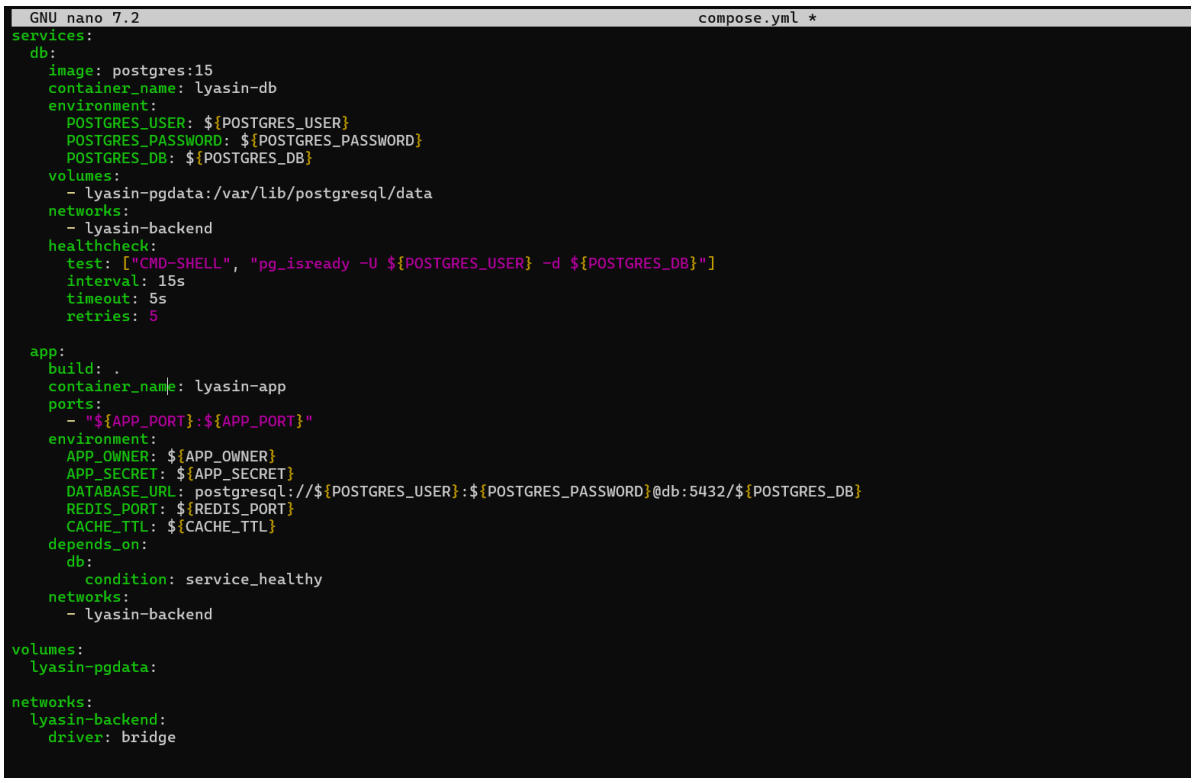


Рисунок 2 — Создание файла `compose.yml`

Проверим логи приложения и базы данных (рисунки 3 и 4).

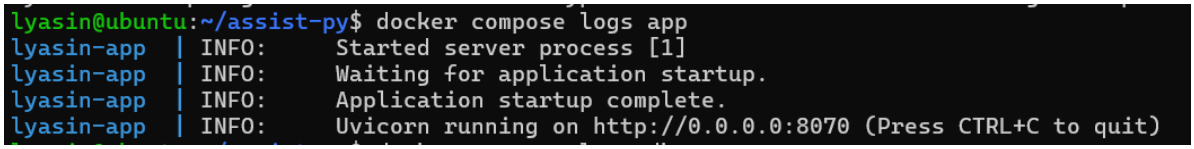


Рисунок 3 — Логи приложения

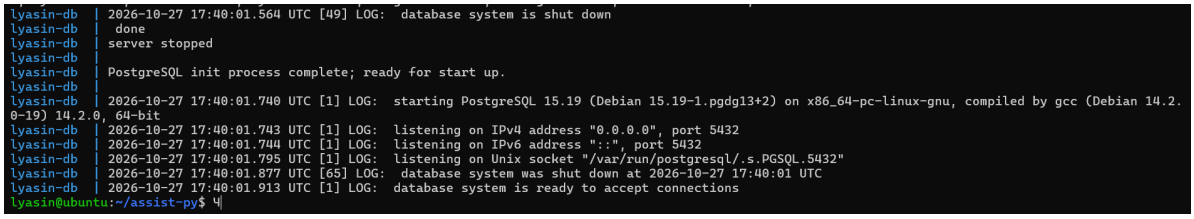


Рисунок 4 — Логи базы данных

Убедимся, что сервис успешно поднялся (рисунок 5).

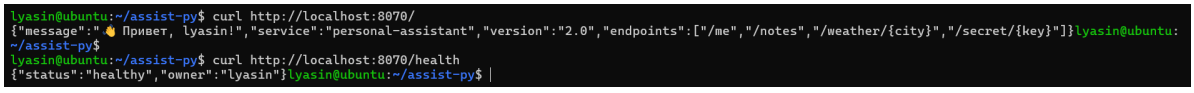


Рисунок 5 — Проверка успешного запуска сервиса

1.2 Этап 2. Добавление Redis для кэширования погоды

Добавим Redis в проект, чтобы кэшировать запросы к внешнему API. Пропишем новый сервис `redis` и добавим его в зависимости сервиса `app` (рисунок 6).

```
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - lyasin-backend

  redis:
    image: redis:7-alpine
    container_name: lyasin-redis
    ports:
      - "${REDIS_PORT}:6379"
    networks:
      - lyasin-backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
```

Рисунок 6 — Добавление сервиса Redis и зависимостей

Файл `main.py` устарел, поэтому в него были внесены изменения для работы кэширования. Перезапустим Compose и проверим, что все три сервиса подняты и имеют статус `healthy` (кроме `app`, для него healthcheck не прописан), рисунок 7.

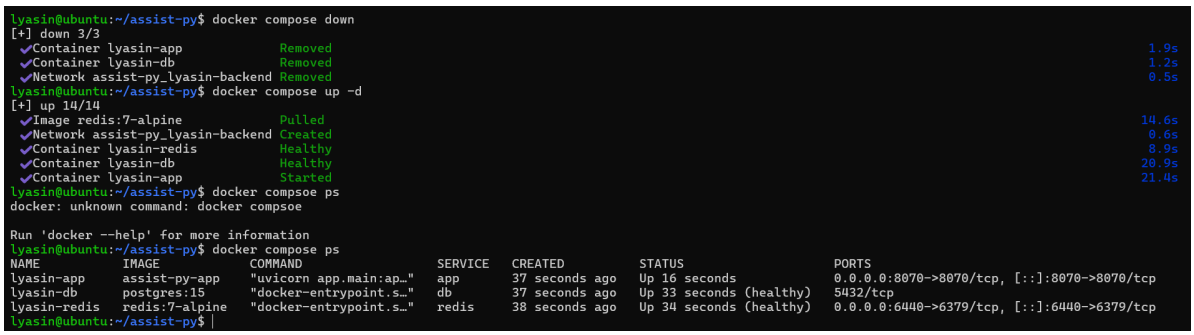


Рисунок 7 — Статусы сервисов после перезапуска

Проверим порядок запуска сервисов через логи: `app` запускается последним (рисунок 8).

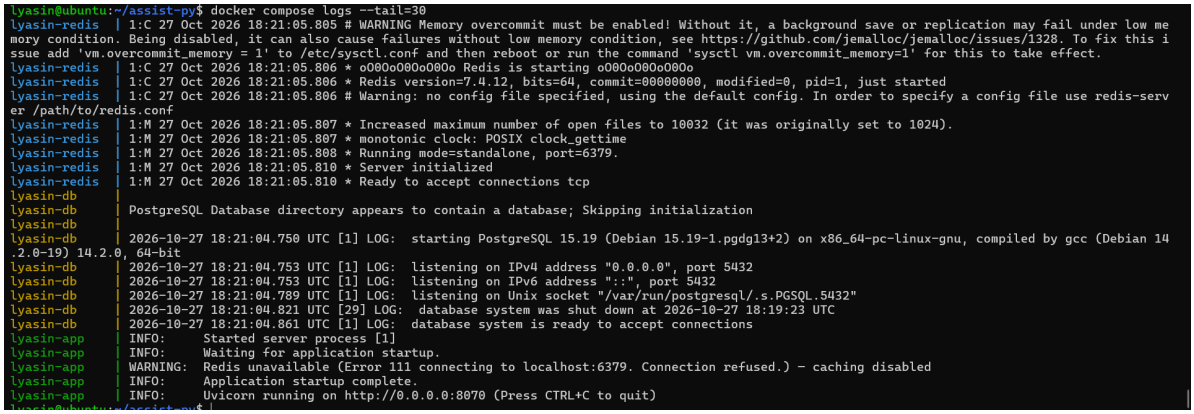


Рисунок 8 — Порядок запуска сервисов по логам

На этом этапе обнаружено, что подключение к Redis не срабатывает. В `compose.yml` добавлена строка, передающая переменную окружения `REDIS_URL` в `app` (рисунок 9).

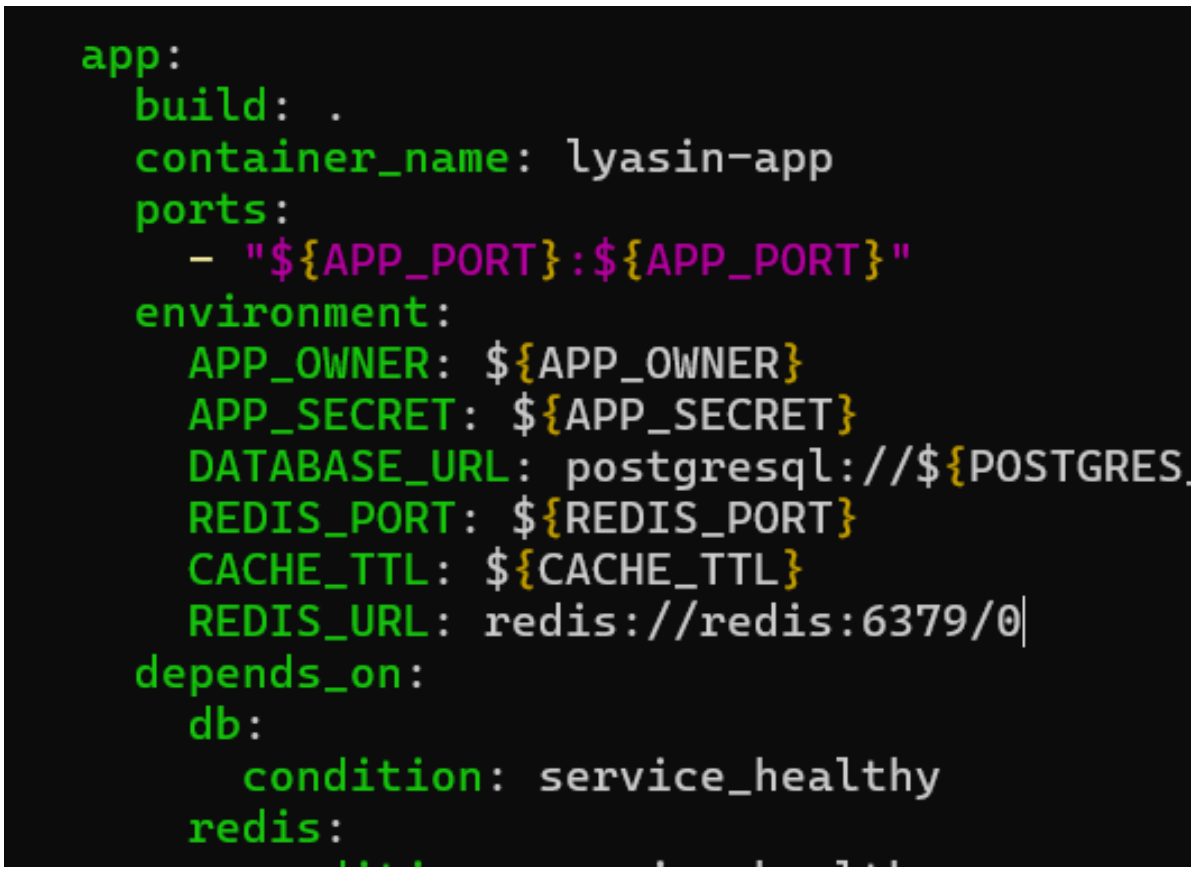


Рисунок 9 — Передача `REDIS_URL` в приложение

Выведем лог `app` и убедимся, что подключение к Redis успешно (рисунок 10).

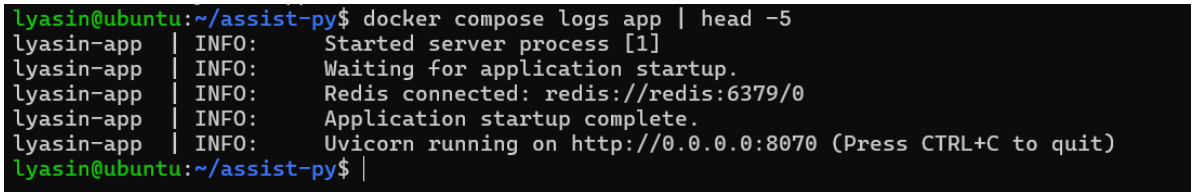


Рисунок 10 — Успешное подключение к Redis

Также `main.py` был отредактирован для использования зеркал, так как исходный API оказался недоступен. Выполним два запроса к приложению: второй произошёл заметно быстрее, в поле `source` первого запроса указано `api`, второго — `cache` (рисунок 11).

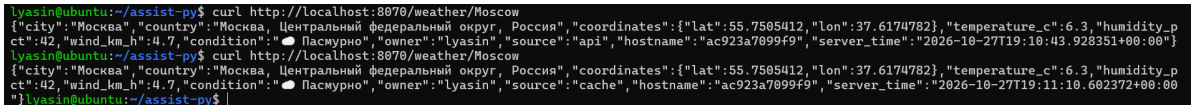


Рисунок 11 — Два запроса: источник `api` и `cache`

Проверим статистику кэша (рисунок 12) и Redis напрямую — убедимся в наличии записи (рисунок 13).



Рисунок 12 — Статистика кэша

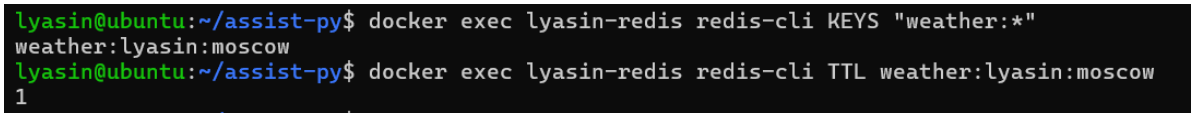


Рисунок 13 — Запись в Redis, полученная напрямую

Посмотрим список сетей: созданная сеть имеет приставку `assist-py_` (рисунок 14). Информация об этой сети показывает, что все три контейнера принадлежат ей (рисунок 15).

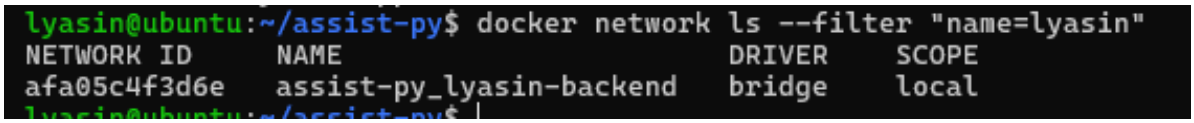


Рисунок 14 — Список сетей



Рисунок 15 — Информация о сети проекта

Аналогично имя volume также имеет приставку `assist-py_` (рисунок 16).

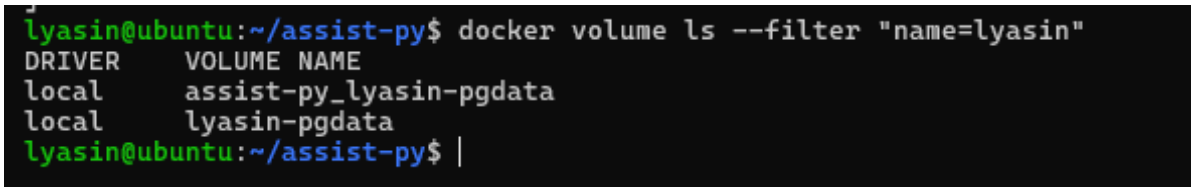


Рисунок 16 — Именованный volume проекта

1.3 Этап 3. Масштабирование сервиса

Закоммитим текущий `compose.yml` и обновим его, добавив возможность масштабирования (рисунок 17).

```yaml
services:
  db:
    image: postgres:15
    container_name: lyasin-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - lyasin-pgdata:/var/lib/postgresql/data
    networks:
      - lyasin-backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: lyasin-redis
    networks:
      - lyasin-backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "${APP_PORT}"
    environment:
      APP_OWNER: ${APP_OWNER}
      APP_SECRET: ${APP_SECRET}
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      REDIS_URL: redis://redis:6379/0
      CACHE_TTL: ${CACHE_TTL}
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - lyasin-backend

volumes:
  lyasin-pgdata:

networks:
  lyasin-backend:
    driver: bridge
```

Рисунок 17 — Обновлённый `compose.yml` с поддержкой масштабирования

Остановим текущие контейнеры и запустим обновлённый Compose с количеством экземпляров `app = 3`. Каждому из трёх контейнеров приложения назначен свой порт из диапазона 32768–32770 (рисунок 18).

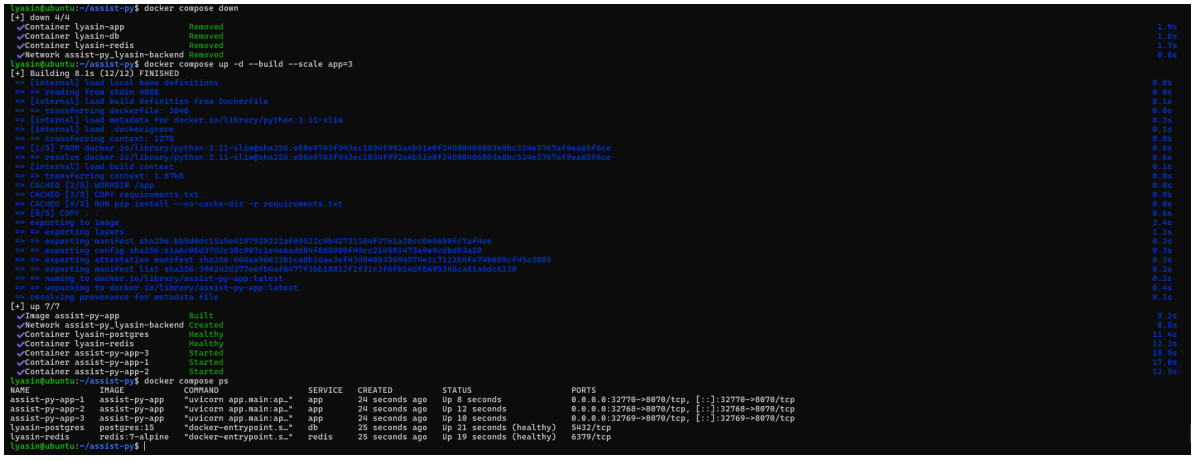


Рисунок 18 — Запуск трёх экземпляров приложения

Проверим, что каждый из сервисов доступен по отдельности и что все экземпляры разделяют общий кэш (рисунок 19).

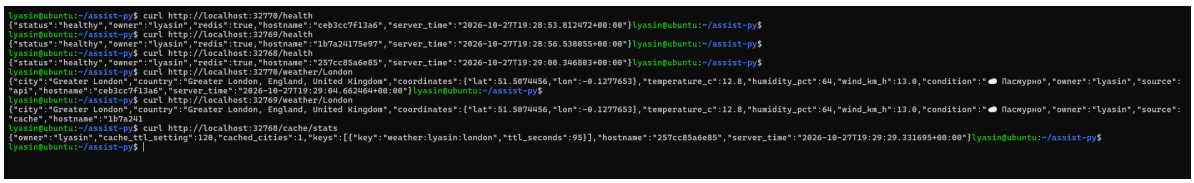


Рисунок 19 — Доступность экземпляров и общий кэш

После проверки вернём предыдущую конфигурацию без масштабирования.

1.4 Этап 4. Финальный артефакт

Финальный артефакт лабораторной работы приведён на рисунке 20.

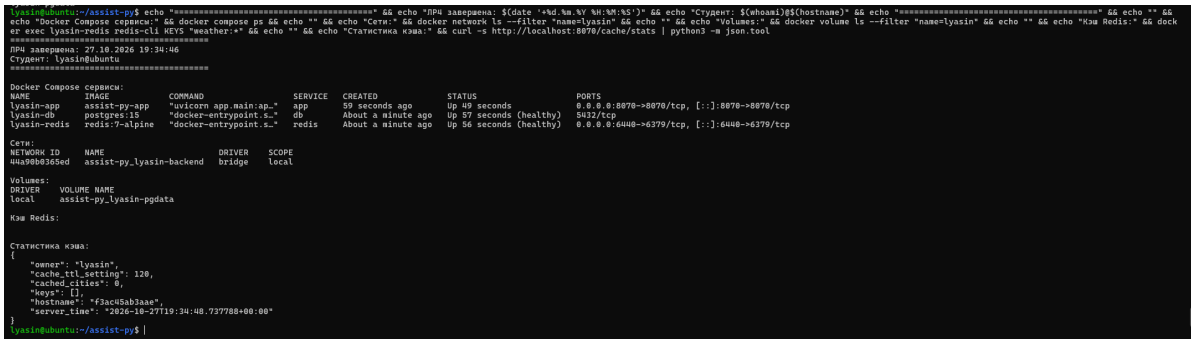


Рисунок 20 — Финальный артефакт лабораторной работы

ЗАКЛЮЧЕНИЕ

В ходе работы запуск приложения, PostgreSQL и Redis были выполнены в Docker Compose. настроены зависимости и порядок запуска. Добавлен Redis для кэширования запросов к внешнему API. Проверено совместное использование сети и именованного volume сервисами проекта, а также выполнено масштабирование приложения до трёх экземпляров с общим кэшем.